



UNIVERSIDAD
DE GRANADA

UNIVERSIDAD DE GRANADA

Escuela Técnica Superior de Ingenierías Informática y de
Telecomunicación (ETSIIT)

Sistema de Información Meteorológica — MeteoRMI

Autor:

Adrián Macías Caballero

Paula Genaro Soria

Asignatura:

Desarrollo de Aplicaciones

en Red

22 de abril de 2026

Índice

1. Diseño de la Interfaz Remota	2
1.1. MeteoService — Interfaz principal del servidor	2
1.2. MeteoCallback — Interfaz de retrollamada del cliente	2
2. Decisiones de Diseño Adoptadas	2
2.1. Patrón Observer para las suscripciones	2
2.2. Excepción remota tipada (MeteoException)	3
2.3. Objeto de valor WeatherData	3
2.4. ConcurrentHashMap para el estado compartido	3
2.5. ScheduledExecutorService para el bucle de actualización	3
3. Reflexión Técnica	3
3.1. Transformación del protocolo en métodos	3
3.2. Problemas que han desaparecido respecto a sockets	4
3.3. Problemas nuevos introducidos por RMI	4
4. Instrucciones de Ejecución	5
4.1. Prerequisitos	5
4.2. Estructura de carpetas	5
4.3. Compilación	5
4.4. Ejecución del servidor	5
4.5. Ejecución del cliente	6
4.6. Ejecución en máquinas distintas	6
4.7. Captura con Wireshark	6

1. Diseño de la Interfaz Remota

El sistema MeteoRMI expone dos interfaces remotas que definen el contrato entre cliente y servidor. Cualquier objeto que implemente estas interfaces puede ser invocado a través de la red de forma transparente, sin que el llamante necesite conocer su ubicación física.

1.1. MeteoService — Interfaz principal del servidor

Contiene las cuatro operaciones del servicio meteorológico, directamente equivalentes a los comandos del protocolo JSON de la Práctica 1:

Método	Retorno	Descripción	Equivalencia P1
getWeather(city)	WeatherData	Consultar clima actual de una ciudad	GET
listCities()	List<String>	Obtener catálogo de ciudades disponibles	LIST
subscribe(city, callback)	WeatherData	Suscribirse a alertas; devuelve estado inicial	SUB
unsubscribe(callback)	void	Cancelar todas las suscripciones activas	UNSUB

Tabla 1: Operaciones del servicio MeteoService

Todas las operaciones lanzan RemoteException (obligatorio en RMI) y las que implican lógica de negocio lanzan además MeteoException, una excepción personalizada serializable que viaja por la red igual que cualquier objeto Java.

1.2. MeteoCallback — Interfaz de retrollamada del cliente

Implementada por el cliente, permite al servidor notificar cambios sin que el cliente tenga que preguntar activamente. Este patrón Observer sobre RMI elimina el hilo de lectura continua que existía en la Práctica 1:

```
void onWeatherChange(WeatherData data) throws RemoteException
```

WeatherData es un objeto serializable (implementa Serializable) que encapsula temperatura, humedad, presión y velocidad del viento. Viaja completo por la red como parte del mecanismo de marshalling de RMI.

2. Decisiones de Diseño Adoptadas

2.1. Patrón Observer para las suscripciones

En la P1, las suscripciones se gestionaban manteniendo un socket abierto por cliente y enviando mensajes JSON cuando el servidor detectaba cambios. En RMI, el cliente registra su propia referencia remota (MeteoCallback) en el servidor. Esto

invierte el control: el servidor llama al cliente en lugar de que el cliente escuche pasivamente.

2.2. Excepción remota tipada (MeteoException)

En lugar de códigos numéricos de estado HTTP (404, 500...) dentro de un JSON, se utiliza una excepción Java que extiende `Exception` e implementa `Serializable`. El compilador obliga al cliente a capturarla, haciendo el manejo de errores explícito y verificable en tiempo de compilación.

2.3. Objeto de valor WeatherData

Los datos meteorológicos se encapsulan en un objeto tipado en lugar de usar `Map<String,Object>` o cadenas JSON. Esto aporta validación en compilación, autocompletado en el IDE y facilita la evolución del modelo sin romper el contrato de la interfaz.

2.4. ConcurrentHashMap para el estado compartido

RMI crea un hilo por cada invocación remota entrante. Para evitar condiciones de carrera al acceder a las estructuras de suscripciones y estados de ciudad, se usan `ConcurrentHashMap`, que ofrecen acceso atómico sin bloqueo explícito en la mayoría de operaciones.

2.5. ScheduledExecutorService para el bucle de actualización

El hilo periódico de actualización (equivalente a `update_loop` en Python) se implementa con `ScheduledExecutorService` en lugar de `Thread+sleep`, lo que garantiza ejecución a intervalos regulares independientemente del tiempo que tarde cada consulta a la API.

3. Reflexión Técnica

3.1. Transformación del protocolo en métodos

El protocolo JSON de la P1 definía una capa de mensajería explícita: el cliente construía objetos como `{command:"GET", city:"Granada"}`, los serializaba, los enviaba por el socket, y el servidor los parseaba. Cualquier campo mal escrito o ausente producía errores silenciosos en tiempo de ejecución.

Con RMI, este protocolo se convierte en firmas de métodos Java. La traducción es directa:

1. El campo `command` desaparece: cada comando es un método con nombre.
2. Los campos de datos se convierten en parámetros tipados.

3. La respuesta JSON se convierte en el tipo de retorno del método.
4. Los códigos de error se convierten en excepciones verificadas.

El compilador verifica ahora aspectos que antes solo fallaban en ejecución: tipos de parámetros, nombre de operaciones, formato de la respuesta.

3.2. Problemas que han desaparecido respecto a sockets

1. Serialización manual: no hay `json.dumps/json.loads` ni gestión de codificación UTF-8.
2. Delimitación de mensajes: el `\n` al final de cada JSON era necesario para que el receptor supiera dónde terminaba el mensaje. RMI gestiona la framing de mensajes internamente.
3. Hilo de lectura en el cliente eliminado: el cliente Python necesitaba un thread daemon permanente leyendo del socket para recibir notificaciones. El callback RMI elimina este hilo.
4. Gestión de múltiples mensajes en un `recv` eliminada: el cliente debía hacer `split(\n)` porque un `recv` podía contener varios mensajes. RMI garantiza que cada invocación llega íntegra.
5. Protocolo ad-hoc eliminado: cualquier cambio en el protocolo JSON requería actualizar el cliente y el servidor de forma coordinada y manual. Con RMI, la interfaz es el contrato y el compilador detecta incompatibilidades.

3.3. Problemas nuevos introducidos por RMI

1. Latencia oculta: una llamada a `server.getWeather("Madrid")` parece local pero puede tardar cientos de milisegundos o fallar con `RemoteException`. El programador debe asumir que toda invocación puede bloquearse o fallar.
2. Acoplamiento tecnológico: tanto cliente como servidor deben estar escritos en Java y compartir las clases de la interfaz y los objetos serializables. Con sockets, cualquier lenguaje podía consumir la API.
3. Problema del callback en NAT/firewall: para que el servidor pueda invocar `onWeatherChange()` en el cliente, el cliente debe ser alcanzable en red (tener un puerto abierto). Esto puede fallar detrás de NAT o firewalls corporativos, mientras que con sockets el cliente siempre iniciaba la conexión.
4. Registro RMI como punto único de fallo: el cliente debe localizar primero el registro RMI (puerto 1099) para obtener la referencia al servicio. Si el registro no está disponible, la conexión falla incluso si el servidor está activo.
5. Gestión de stubs obsoletos: si el servidor se reinicia, los stubs (referencias remotas) que el cliente tenía en memoria quedan inválidos y producen `RemoteException`, obligando al cliente a reconectarse.

6. Mayor verbosidad: la necesidad de declarar `RemoteException` en cada método, implementar `UnicastRemoteObject` y gestionar el registro RMI añade código boilerplate significativo respecto a la solución con sockets.

4. Instrucciones de Ejecución

4.1. Prerequisitos

1. Java JDK 11 o superior instalado y en el PATH.
2. Visual Studio Code con Extension Pack for Java.
3. Conexión a Internet para descargar json-simple (solo la primera vez).
4. Wireshark instalado para la captura de tráfico (opcional, análisis de red).

4.2. Estructura de carpetas

```
MeteoRMI/  
  src/meteo/  
  lib/  
  bin/  
  compile.bat  
  run_server.bat  
  run_client.bat
```

4.3. Compilación

Abrir una terminal integrada en VSCode (Ctrl+ñ) en la carpeta raíz del proyecto y ejecutar:

```
.\compile.bat
```

El script descarga automáticamente json-simple la primera vez y compila todos los .java. Al finalizar muestra: Compilacion completada.

4.4. Ejecución del servidor

En una primera terminal de VSCode:

```
.\run_server.bat
```

El servidor muestra el mensaje de bienvenida e indica que está escuchando en el puerto 1099 (registro RMI).

4.5. Ejecución del cliente

En una segunda terminal de VSCode (manteniendo la primera abierta):

```
.\run_client.bat
```

El cliente solicita la IP del servidor. Pulsar Enter para usar localhost. Una vez conectado, los comandos disponibles son:

1. list — muestra las ciudades disponibles
2. get <ciudad> — consulta el clima actual (ej: get Granada)
3. sub <ciudad> — suscribe a alertas automáticas (ej: sub Madrid)
4. unsub — cancela todas las suscripciones
5. salir — desconecta y cierra el cliente

4.6. Ejecución en máquinas distintas

Para demostrar conectividad real entre dos equipos de la red local:

- En el servidor, editar `MeteoServer.java` y cambiar host a `0.0.0.0` para aceptar conexiones externas.
- Establecer la propiedad `java.rmi.server.hostname` a la IP real del servidor antes de arrancar (añadir `-Djava.rmi.server.hostname=<IP_SERVIDOR>` en `run_server.bat`).
- En el cliente, introducir la IP del servidor cuando se solicite al arrancar.
- Abrir el puerto 1099 en el firewall de Windows (Panel de control → Firewall → Regla de entrada).

4.7. Captura con Wireshark

Para analizar el tráfico RMI con Wireshark:

- Abrir **Wireshark** y seleccionar la interfaz de red activa (o Loopback para pruebas locales).
- Aplicar el filtro: `tcp.port == 1099`
- Ejecutar cliente y servidor, realizar operaciones (`get`, `sub`, ...).
- Observar los paquetes etiquetados como **JRMI**. A diferencia del JSON legible de la P1, el contenido aparece como datos binarios serializados (**Java Object Serialization**), lo que evidencia que RMI maneja la serialización de forma opaca.